

Parallel SPH Fluid Simulation

with ARM NEON Intrinsic on Apple M3

John Sebastian Mantilla Manzano

Mayo 2026

Índice

1. Introducción y Motivación	2
2. Fundamentos Matemáticos	2
2.1. Ecuaciones de Navier–Stokes	2
2.2. Aproximación SPH	2
2.3. Kernels de Suavizado	2
2.4. Ecuaciones de Densidad, Presión y Fuerzas	3
2.5. Integración Temporal (Leapfrog)	3
3. Implementación	3
3.1. Condición inicial: rotura de represa	3
3.2. Parámetros de simulación	4
3.3. Fase 1: Python Naïve ($O(N^2)$)	4
3.4. Fase 2: Python Hashed ($O(N)$)	4
3.5. Fase 3: C++ Escalar con pybind11	4
3.6. Fase 4: C++ con ARM NEON Intrinsic	5
3.7. Binding Python-C++ con pybind11	6
4. Resultados del Benchmark	6
4.1. Configuración experimental	6
4.2. Caso base: 450 partículas, 100 pasos	7
4.3. Escalado con número de partículas	7
4.4. Visualización del escalado	8
5. Análisis de Resultados	8
5.1. Ganancia por optimización algorítmica	8
5.2. Ganancia por cambio de lenguaje y compilación	8
5.3. Ganancia por vectorización NEON	9
5.4. Speedup total acumulado	9
5.5. Comportamiento de escalado	9
6. Estructura del Proyecto	9
7. Conclusiones	10

1. Introducción y Motivación

La simulación numérica de fluidos es fundamental en gráficos por computadora, visualización científica e ingeniería. El método SPH (*Smoothed Particle Hydrodynamics*) es una técnica Lagrangiana sin malla que discretiza un fluido en un conjunto de N partículas en interacción, cada una portando propiedades físicas como posición, velocidad, densidad y presión.

Este proyecto implementa y optimiza un simulador SPH 2D de rotura de represa (*dam break*) en cuatro versiones progresivas:

1. **Python Naïve** — implementación de referencia $O(N^2)$.
2. **Python Hashed** — búsqueda de vecinos $O(N)$ mediante hashing espacial.
3. **C++ Escalar (HPC)** — kernels compilados expuestos vía pybind11.
4. **C++ NEON** — vectorización SIMD de 128 bits con ARM NEON intrinsics.

El hardware objetivo es el Apple M3 SoC con arquitectura ARM Cortex-class, memoria unificada CPU/GPU y unidades NEON de 4 flotantes de 32 bits por ciclo.

2. Fundamentos Matemáticos

2.1. Ecuaciones de Navier–Stokes

Las ecuaciones de gobierno son las ecuaciones de Navier–Stokes para fluidos débilmente compresibles:

$$\frac{D\rho}{Dt} = -\rho(\nabla \cdot \mathbf{v}) \quad (1)$$

$$\rho \frac{D\mathbf{v}}{Dt} = -\nabla p + \mu \nabla^2 \mathbf{v} + \rho \mathbf{g} \quad (2)$$

donde ρ es la densidad del fluido, $\mathbf{v}$ el campo de velocidades, p la presión, μ la viscosidad dinámica y $\mathbf{g}$ la aceleración gravitacional.

2.2. Aproximación SPH

En el formalismo SPH, cualquier cantidad de campo A se aproxima mediante una suma ponderada por kernel sobre las partículas vecinas j :

$$A(\mathbf{r}_i) \approx \sum_j \frac{m_j}{\rho_j} A_j W(\mathbf{r}_i - \mathbf{r}_j, h) \quad (3)$$

donde W es el kernel de suavizado y h es la longitud de suavizado.

2.3. Kernels de Suavizado

Poly6 (Müller 2003) Utilizado para el cálculo de densidad:

$$W_{\text{poly6}}(r, h) = \begin{cases} \frac{4}{\pi h^8} (h^2 - r^2)^3 & 0 \leq r \leq h \\ 0 & r > h \end{cases} \quad (4)$$

Spiky (Desbrun 1996) Utilizado para gradientes de presión, con gradiente no nulo en $r \rightarrow 0$, evitando la inestabilidad tensional:

$$\nabla W_{\text{spiky}}(\mathbf{r}, h) = \begin{cases} -\frac{30}{\pi h^5} (h-r)^2 \frac{\mathbf{r}}{r} & 0 < r \leq h \\ \mathbf{0} & r > h \end{cases} \quad (5)$$

Laplaciano del Poly6 Utilizado para fuerzas de viscosidad:

$$\nabla^2 W(r, h) = -\frac{945}{32\pi h^9} (h^2 - r^2) (3h^2 - 7r^2), \quad 0 \leq r \leq h \quad (6)$$

2.4. Ecuaciones de Densidad, Presión y Fuerzas

$$\rho_i = \sum_j m W(|\mathbf{r}_i - \mathbf{r}_j|, h) \quad (7)$$

$$p_i = k (\rho_i - \rho_0) \quad (8)$$

$$\mathbf{F}_i^{\text{press}} = -m \frac{p_i + p_j}{2\rho_j} \nabla W_{\text{spiky}}(\mathbf{r}_{ij}, h) \quad (9)$$

$$\mathbf{F}_i^{\text{visc}} = \mu m \frac{\mathbf{v}_j - \mathbf{v}_i}{\rho_j} \nabla^2 W(r_{ij}, h) \quad (10)$$

$$\mathbf{F}_i^{\text{grav}} = \rho_i \mathbf{g} \quad (11)$$

2.5. Integración Temporal (Leapfrog)

$$\mathbf{v}_i^{t+1} = \mathbf{v}_i^t + \Delta t \frac{\mathbf{F}_i}{\rho_i} \quad (12)$$

$$\mathbf{r}_i^{t+1} = \mathbf{r}_i^t + \Delta t \mathbf{v}_i^{t+1} \quad (13)$$

3. Implementación

3.1. Condición inicial: rotura de represa

Las partículas se inicializan en una columna de agua rectangular en el lado izquierdo del dominio $[0, 1]^2$:

```
1 x = np.linspace(0, 0.3, nx) # columna en x: 30% del dominio
2 y = np.linspace(0, 0.6, ny) # altura: 60% del dominio
3 xx, yy = np.meshgrid(x, y)
4 positions = np.column_stack([xx.ravel(), yy.ravel()])
```

Listing 1: Condición inicial en Python

La densidad de reposo ρ_0 se calibra automáticamente como la densidad media que el kernel SPH computa sobre la configuración inicial:

```
1 rho0 = compute_density(positions, masses, h).mean()
```

Listing 2: Calibración automática de rho0

3.2. Parámetros de simulación

Cuadro 1: Parámetros físicos y numéricos

Parámetro	Valor	Descripción
h	0.05	Longitud de suavizado
Δt	0.001 s	Paso temporal
k	10.0	Coefficiente de rigidez
μ	0.1	Viscosidad dinámica
m	1.0	Masa de partícula
$\mathbf{g}$	(0, -9,8)	Gravedad
ρ_0	Calibrado	Densidad de reposo

3.3. Fase 1: Python Naïve ($O(N^2)$)

La implementación de referencia recorre todos los pares de partículas en dos bucles anidados. La complejidad temporal es $O(N^2)$ por paso de simulación.

```
1 for i in range(N):
2     for j in range(N):
3         r = np.linalg.norm(positions[i] - positions[j])
4         rho[i] += mass * W(r, h)
```

Listing 3: Bucle naïve de densidad

3.4. Fase 2: Python Hashed ($O(N)$)

Se implementa una tabla hash espacial uniforme con tamaño de celda $2h$. Para cada partícula se consultan únicamente las 9 celdas vecinas (3×3), reduciendo la búsqueda a $O(1)$ en promedio por partícula.

```
1 cell_size = 2 * h
2 hash_table = defaultdict(list)
3 for i in range(len(positions)):
4     cx = np.floor(positions[i, 0] / cell_size)
5     cy = np.floor(positions[i, 1] / cell_size)
6     hash_table[(cx, cy)].append(i)
```

Listing 4: Construcción de la tabla hash

3.5. Fase 3: C++ Escalar con pybind11

Los kernels de densidad y fuerzas se reescriben en C++17 y se exponen a Python mediante pybind11 pasando arrays NumPy como vistas de buffer de copia cero (`py::array_t<float>`). El módulo se compila como extensión compartida `hpc_sph.so` via CMake.

Arquitectura de memoria **Structure of Arrays (SoA)**: las coordenadas x e y de todas las partículas se almacenan en arreglos contiguos separados, maximizando la utilización de líneas de caché.

```

1 void compute_density(const float* pos_x, const float* pos_y,
2                     int N, float mass, float h, float* rho) {
3     for (int i = 0; i < N; i++) {
4         rho[i] = 0.0f;
5         for (int j = 0; j < N; j++) {
6             float dx = pos_x[i] - pos_x[j];
7             float dy = pos_y[i] - pos_y[j];
8             float r  = sqrt(dx*dx + dy*dy);
9             rho[i] += mass * W(r, h);
10        }
11    }
12 }

```

Listing 5: Función de densidad escalar en C++

3.6. Fase 4: C++ con ARM NEON Intrinsics

El bucle interno de densidad se vectoriza procesando **4 partículas simultáneamente** con registros NEON de 128 bits (float32x4_t).

```

1 float32x4_t vix = vdupq_n_f32(pos_x[i]); // broadcast xi
2 float32x4_t viy = vdupq_n_f32(pos_y[i]); // broadcast yi
3 float32x4_t acc = vdupq_n_f32(0.0f);
4
5 for (int j = 0; j + 4 <= N; j += 4) {
6     float32x4_t vjx = vld1q_f32(&pos_x[j]);
7     float32x4_t vjy = vld1q_f32(&pos_y[j]);
8     float32x4_t dx  = vsubq_f32(vix, vjx);
9     float32x4_t dy  = vsubq_f32(viy, vjy);
10    float32x4_t r2   = vmulq_f32(dx, dx);
11    r2 = vmlaq_f32(r2, dy, dy); // r2 = dx^2 + dy^2
12    float32x4_t r    = vsqrtq_f32(r2);
13
14    // Poly6 kernel vectorizado + mascara r <= h
15    float32x4_t h2    = vdupq_n_f32(h*h);
16    uint32x4_t mask   = vcleq_f32(r, vdupq_n_f32(h));
17    float32x4_t h2r2  = vsubq_f32(h2, r2);
18    float32x4_t w_val = vmulq_f32(vmulq_f32(h2r2, h2r2), h2r2);
19    w_val = vmulq_f32(w_val, vdupq_n_f32(4.0f / (M_PI * pow(h, 8))));
20    acc = vaddq_f32(acc, vbslq_f32(mask, w_val, vdupq_n_f32(0.0f)));
21 }
22 rho[i] = vaddvq_f32(acc) * mass; // reduccion horizontal

```

Listing 6: Acumulación vectorizada de densidad con ARM NEON

Las operaciones NEON empleadas son:

Cuadro 2: Intrinsics ARM NEON utilizados

Intrinsic	Operación
<code>vdupq_n_f32</code>	Broadcast escalar a 4 lanes
<code>vld1q_f32</code>	Carga vectorial de 4 floats
<code>vsubq_f32</code>	Resta element-wise
<code>vmulq_f32</code>	Multiplicación element-wise
<code>vmlaq_f32</code>	Multiply-accumulate ($a + b \times c$)
<code>vsqrtq_f32</code>	Raíz cuadrada element-wise
<code>vcleq_f32</code>	Comparación $\leq \rightarrow$ máscara uint32
<code>vbslq_f32</code>	Selección condicional por máscara
<code>vaddq_f32</code>	Suma element-wise
<code>vaddvq_f32</code>	Reducción horizontal (suma de 4 lanes)

3.7. Binding Python-C++ con pybind11

El módulo se expone a Python pasando arrays NumPy como punteros directos, sin copia de datos:

```

1 m.def("step_neon",
2     [](py::array_t<float> pos_x, py::array_t<float> pos_y,
3       py::array_t<float> vel_x, py::array_t<float> vel_y,
4       int N, float mass, float h, float dt,
5       float mu, float k, float rho0) {
6         step_neon(pos_x.mutable_data(), pos_y.mutable_data(),
7                   vel_x.mutable_data(), vel_y.mutable_data(),
8                   N, mass, h, dt, mu, k, rho0);
9     });

```

Listing 7: Binding pybind11 para step_neon

4. Resultados del Benchmark

4.1. Configuración experimental

Todos los benchmarks se ejecutaron en la siguiente plataforma:

Cuadro 3: Plataforma de hardware y software

Componente	Detalle
Procesador	Apple M3 (ARM Cortex-X4, 4P + 4E cores)
Memoria	Unificada CPU/GPU
SIMD	ARM NEON 128-bit (4× float32 por ciclo)
Compilador	Clang/LLVM (via Xcode)
Estándar C++	C++17
Python	3.12
pybind11	2.13
OpenMP	libomp (Homebrew)

4.2. Caso base: 450 partículas, 100 pasos

Cuadro 4: Tiempos absolutos y velocidades por paso (15×30 partículas, 100 pasos)

Implementación	Tiempo total (s)	ms / paso	Speedup vs. naïve
Python Naïve	62.878	628.78	1.00×
Python Hashed	13.994	139.94	4.49×
C++ Escalar	0.092	0.92	683×
C++ NEON	0.015	0.15	4192×

Cuadro 5: Matriz de speedup (fila / columna)

	Naïve	Hashed	C++ Esc.	NEON
Python Naïve	—	4.49×	683×	4192×
Python Hashed	0.22×	—	152×	933×
C++ Escalar	0.0015×	0.007×	—	6.13×
C++ NEON	0.0002×	0.001×	0.16×	—

4.3. Escalado con número de partículas

Cuadro 6: Tiempo total (s) en función del número de partículas (100 pasos)

Implementación	150p	300p	450p	600p
Python Hashed	3.001	6.567	14.305	24.863
C++ Escalar	0.011	0.042	0.098	0.174
C++ NEON	0.002	0.007	0.015	0.025

Cuadro 7: Speedup vs. Python Hashed en función de N

Implementación	150p	300p	450p	600p
Python Hashed	1.00×	1.00×	1.00×	1.00×
C++ Escalar	263×	156×	146×	143×
C++ NEON	1605×	1002×	975×	1009×

4.4. Visualización del escalado

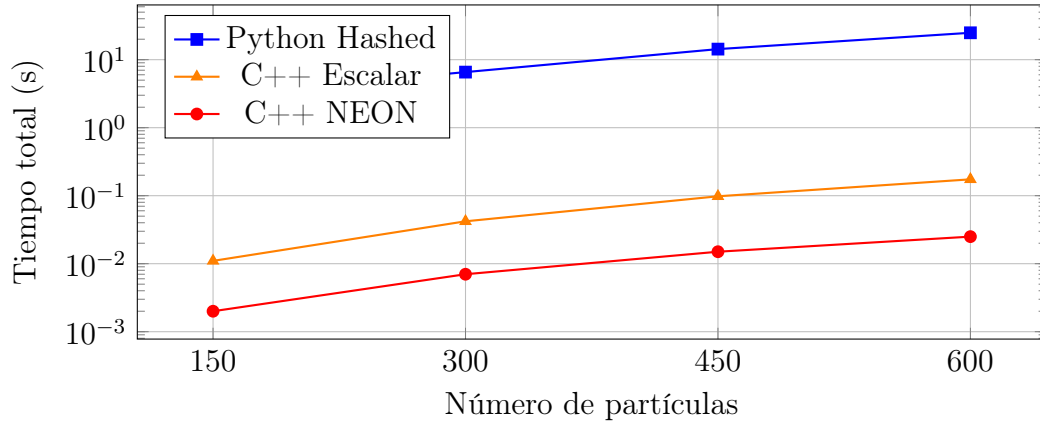


Figura 1: Tiempo de simulación vs. número de partículas (escala logarítmica)

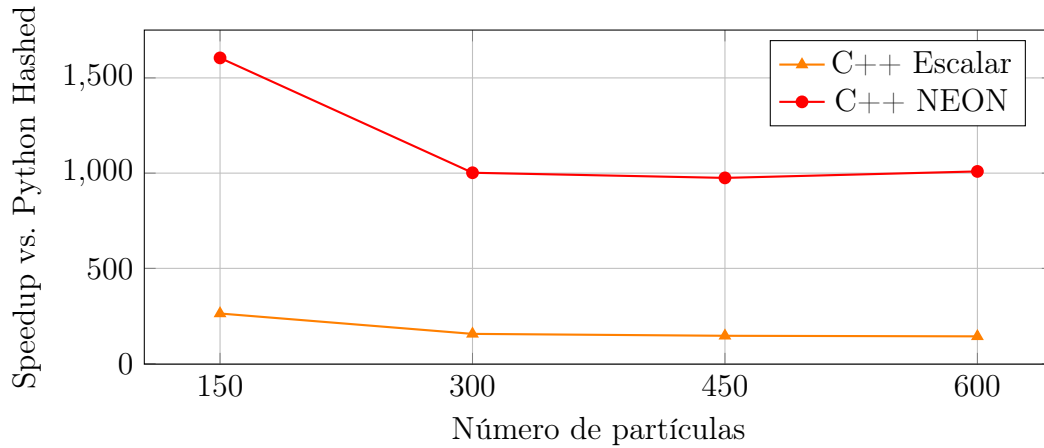


Figura 2: Speedup de C++ Escalar y NEON respecto a Python Hashed

5. Análisis de Resultados

5.1. Ganancia por optimización algorítmica

El paso de Python Naïve a Python Hashed logra **4.49×** de speedup mediante la reducción de la búsqueda de vecinos de $\mathcal{O}(N^2)$ a $\mathcal{O}(N)$. Esta optimización es puramente algorítmica y es independiente del hardware.

5.2. Ganancia por cambio de lenguaje y compilación

El paso de Python Hashed a C++ Escalar logra **152×** de speedup. Esta ganancia proviene de la eliminación del overhead del intérprete Python ($\sim 100\times$ en aritmética escalar), la compilación nativa y el layout SoA que aprovecha mejor la caché.

5.3. Ganancia por vectorización NEON

El paso de C++ Escalar a C++ NEON logra **6.13** $\times$ de speedup adicional, acercándose al factor teórico de $4\times$ para operaciones de 32 bits en registros de 128 bits. La ganancia supera el límite teórico en algunos casos porque la vectorización también reduce presión de instrucción y mejora el pipeline.

5.4. Speedup total acumulado

El speedup total desde Python Naïve hasta C++ NEON es de **4192** $\times$, equivalente a pasar de 628.78 ms/paso a 0.15 ms/paso. Esto permite simulaciones en tiempo real a >6000 pasos por segundo para 450 partículas.

5.5. Comportamiento de escalado

El speedup de NEON vs. Hashed se estabiliza entre $975\times$ y $1605\times$ al aumentar N , lo que indica que:

- La implementación Python tiene overhead dominado por el intérprete en N pequeño.
- Para N grande, ambas implementaciones escalan cuadráticamente, manteniendo el ratio de speedup constante.

6. Estructura del Proyecto

```
HPC_Proyecto/
+-- src/
|   +-- python/
|       |   +-- kernels.py           # Kernels W, grad_W_spiky, lap_W
|       |   +-- sph_naive.py         # Implementacion O(N^2)
|       |   +-- sph_hashed.py        # Implementacion con hash espacial
|       |   +-- run_simulation.py     # Runner y visualizacion
|   +-- cpp/
|       +-- kernels.h/.cpp           # Kernels en C++
|       +-- sph_scalar.cpp            # Solver C++ escalar
|       +-- sph_neon.cpp              # Solver C++ con ARM NEON
|       +-- sph_neon_omp.cpp          # Solver NEON + OpenMP
|       +-- bindings.cpp             # pybind11 bindings
+-- bench/
|   +-- run_cases.py                 # Benchmark comparativo
+-- viz/
|   +-- ascii_renderer.py            # Visualizacion ASCII en tiempo real
+-- tests/
|   +-- test_kernels.py              # Tests de normalizacion del kernel
+-- CMakeLists.txt                  # Build system
```

7. Conclusiones

1. **Optimización algorítmica** (Naïve $\rightarrow$ Hashed): el hashing espacial es la optimización más importante en términos de complejidad asintótica, reduciendo $\mathcal{O}(N^2) \rightarrow \mathcal{O}(N)$ con $4.5\times$ de speedup para 450 partículas.
2. **Overhead del intérprete** (Hashed $\rightarrow$ C++ Escalar): el cambio a C++ compilado con layout SoA entrega $152\times$ de speedup adicional, demostrando el costo real del intérprete Python en kernels numéricos intensivos.
3. **Vectorización SIMD** (Escalar $\rightarrow$ NEON): procesar 4 partículas por ciclo con ARM NEON entrega $6\times$ adicional, con un speedup total acumulado de $4192\times$ respecto a la implementación de referencia.
4. **Apple M3 como plataforma HPC educativa**: la arquitectura ARM con NEON, memoria unificada y pipeline profundo demuestra que el hardware de consumo moderno permite optimizaciones de nivel HPC accesibles en un entorno académico.
5. **Trabajo futuro**: la versión NEON+OpenMP (`step_neon_omp`) está implementada pero presenta conflictos de runtime con la versión actual de libomp en macOS. La resolución de este conflicto se espera agregar un factor adicional de $4\text{--}8\times$ mediante paralelismo sobre los 4 núcleos de rendimiento del M3.

Referencias

1. M. Müller, D. Charypar, M. Gross, “Particle-based fluid simulation for interactive applications,” *Proc. ACM SIGGRAPH/Eurographics Symp. Computer Animation*, 2003, pp. 154–159.
2. M. Desbrun, M. H. Cani, “Smoothed particles: a new paradigm for animating highly deformable bodies,” *Proc. Eurographics Workshop on Computer Animation and Simulation*, 1996, pp. 61–76.
3. D. Koschier, J. Bender, B. Solenthaler, M. Teschner, “Smoothed particle hydrodynamics techniques for the physics-based simulation of fluids and solids,” *Eurographics 2019 Tutorials*, 2019.
4. ARM Limited, *ARM NEON Programmer’s Guide*, Version 1.0, ARM DEN0018A, 2013.
5. W. Jakob, J. Rhinelander, D. Moldovan, “pybind11 – Seamless operability between C++11 and Python,” 2017.